Q & A

Quick Find

Considered	Ref.
IDs, duplicates	1. Keys and duplicates
How the code stops bad data	2. How the code enforces a rule
Where insert, update or delete happens	3. Insert, update and delete
The search or the join	4. Search queries
SQL injection, security, other farms' data	5. Security
Where the data came from, bad rows	6. The data
The advanced functions	7. Advanced functions

1. Keys and Duplicates

IDs that prevent duplication

> Every table has two layers. First, the database assigns each row its own ID, which cannot be typed in or reused. Second, each table has a UNIQUE rule on its real-world key. For a plot, that key is the farm plus the plot name, so one farm cannot have two plots called North field.

REF: plot table
FOCUS: `GENERATED ALWAYS AS IDENTITY PRIMARY KEY` on line 113, then `UNIQUE (farm_id, name)` on line 118.

Keys on the other tables

> Each key matches what makes the record unique in the real world. A catalog entry is one farm plus one plant, so that pair is the primary key. A taxon is its full path from the kingdom down. A product is its slug. No two farms can share a web domain or an owner email.

Table	What can't repeat	REF
crop_catalog	one farm + one plant	schema.sql lines 121 to 130
taxon	the full path, like Plantae/.../Allium cepa	schema.sql lines 28 to 37
product	the slug, always lowercase	schema.sql lines 40 to 45
product_alias	the cleaned auction label, so one label means one product	schema.sql lines 48 to 54
farm	web domain, owner email	schema.sql lines 101 to 109

2. How the Code Enforces a Rule

Where the rules live

> The rules live in the database, not in the Python, so every page and every script is held to them, not just one form. The Python does two things: it rolls back a failed write, and it turns the database error into a message the user can read.

1. The rule, in the schema.

REF: plot_name_unique

1. Every write goes through one function. If PostgreSQL rejects the write, the transaction is rolled back and the error is passed up.

REF: execute() in db.py

1. The name of the broken rule is read from the error and mapped to a plain message.

REF: errors.py
FOCUS: `"plot_name_unique": "This farm already has a plot with that name."`

Adding the same crop twice

> Handled before it errors. The insert uses ON CONFLICT DO NOTHING, so a repeat changes nothing, and the page reports that the crop is already in the catalog.

REF: INSERT_ENTRY_SQL and catalog_add()

"Needs a unit" message origin

> A named CHECK constraint: an event with a quantity must also have a unit.

REF: farm_event_unit_required

Form checks before the database

> Forms check only type and length: a number must be a number, and a name has a maximum length. The data rules stay in the database, so there is one place to change them.

REF: forms.py

3. Insert, Update and Delete

Insert, update and delete code

> Each table the user changes has its own file. The SQL is written by hand at the top of the file, and the page functions below it run that SQL.

Table	SQL	Page code
farm_event	INSERT, UPDATE, DELETE	event_new, event_edit, event_delete
plot	INSERT, UPDATE, DELETE	plot_new, plot_edit, plot_delete
crop_catalog	INSERT, UPDATE, DELETE	catalog_add, catalog_edit, catalog_delete

FOCUS: the farm_event row, which holds the harvest from the demo.

Deleting a plot keeps its harvests

> The foreign key from an event to its plot is ON DELETE SET NULL. Deleting a plot clears the plot on its events instead of deleting them.

REF: farm_event.plot_id

Deleting a farm

> A farm's plots, catalog and events are deleted with it, because those foreign keys are ON DELETE CASCADE. Those rows only have meaning inside that farm.

REF: plot.farm_id

4. Search Queries

Crop search

> Searches three fields at once: scientific name, common name and product title. Results are ranked by how many auction prices each plant has, so the most-traded plant comes first.

REF: CATALOG_SEARCH_SQL

Harvest vs. market join

> Starts from a farm's events and joins seven tables: farm_event, farm, event_kind, taxon, product, product_alias and market_lot. Only event kinds marked as a yield count, and only lots within 30 days of the harvest. The join is on the taxon, not the name, so every onion lot counts however the auction spelled it.

REF: HARVEST_PRICES_SQL
FOCUS: the JOIN lines 92 to 101, then the 30-day window on lines 100 and 101.

Seasonal price curve

> Groups every price for a plant by month across all years, and returns the lowest low, the mean and the highest high. Uses one unit at a time, because a price per peck cannot be averaged with a price per bushel.

REF: PRICE_CURVE_SQL

Home page counts

> Each number is a live COUNT on its table. The match percentage comes from a view.

REF: TABLE_COUNTS_SQL and v_resolution_coverage

5. Security

SQL injection

> Every value is passed as a bound parameter, such as `%(farm_id)s`. PostgreSQL receives the SQL text and the values separately, so user input is always treated as data and never run as SQL.

REF: INSERT_EVENT_SQL, then cur.execute(sql, params)

One farm cannot change another farm's records

> Every UPDATE and DELETE includes `AND farm_id = %(farm_id)s` in its WHERE clause. If a record belongs to a different farm, nothing matches and nothing changes.

REF: UPDATE_EVENT_SQL and DELETE_EVENT_SQL
FOCUS: line 57 and line 62.

Login

> Not yet implemented. Accounts arrive with deployment the week of November 9. Each farm already has a unique owner email in the schema for a login to attach to.

6. The Data

Data sources

> Public sources, collected by the scripts in the loaders folder: the USDA Market News archive for Mount Hope, County Line Produce Auction's market reports, the GBIF plant taxonomy, and OpenStreetMap for Ohio farms. One script downloads the data and another builds the database.

REF: the loaders folder

Bad rows

> Each price is tested against the same rules as the schema before it is loaded. A row that fails goes to the quarantine table with the reason, so nothing is silently dropped. 26 rows were quarantined, for example a cabbage line whose average was above its high.

REF: check_lot() and where it is used

Duplicate farms in the source data

> The loader keeps one farm per name and county, and keeps each web domain only once. The schema backs this up with UNIQUE on the domain.

REF: farm loading

7. Advanced Functions

> Planned for Stage 5 and not yet built. The roll-up and ranking queries have been tested on the loaded data; the pages for them come next.

REF: taxon table
FOCUS: `parent_taxon_id`, the column the recursion follows.

Topic	Detail
Taxonomic price roll-up	The number of levels under a node varies by branch, so a fixed number of joins cannot cover it. A recursive CTE walks any depth in one query. Asparagales: 305 nodes, 1,805 prices.

Topic	Detail
Recursion safety	A CHECK stops a node from being its own parent, every node has a unique path, and depth is limited to 0 through 11.
Crop decision engine	Score = mean price / (1 + price variation), compared within one unit. Requires at least 12 lots across 2 or more years. Ranked with `RANK() OVER`.
Label resolution ladder	Four matching rules. Two equal matches mark the label ambiguous instead of guessing. A person approves every match.
Measure of success	The match rate on the home page rises from 85.8%.